BME513 - Inteligencia Artificial en Salud

Logistic regression

Alejandro Veloz

Logistic regression for classification

Classification predicts discrete values for y (regression predicts continuous values).

Focus on binary classification where $y \in \{0, 1\}$:

- 0 = Negative class (denoted by "-").
- 1 = Positive class (denoted by "+").

Applications generalize to multi-class problems (more than two categories).

(Example) Email spam classification. Training example pair:

Input $\mathbf{x}^{(i)} =$ Features of email. Label $y^{(i)} = 1$ for spam, 0 for non-spam.

Logistic regression for classification

Linear regression is unsuitable for binary classification:

- Produces values outside the range $[0, 1]$.
- Poor performance on discrete $y \in \{0, 1\}$.
- Solution: Modified hypothesis using logistic function.

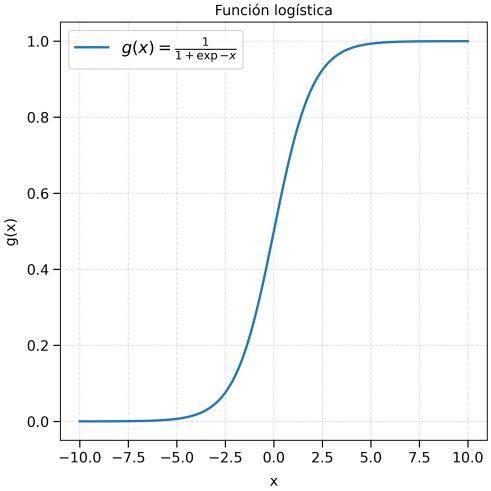
Logistic regression for classification

We define the hypothesis as:

$$h_{\boldsymbol{\theta}}(\mathbf{x}) = g(\boldsymbol{\theta}^\top \mathbf{x}), \quad g(z) = \frac{1}{1 + e^{-z}}.$$

- $h_{\boldsymbol{\theta}}(\mathbf{x}) \in (0, 1)$.
- Can be interpreted as $P(y = 1 \mid \mathbf{x}; \boldsymbol{\theta})$.

Logistic function for classification



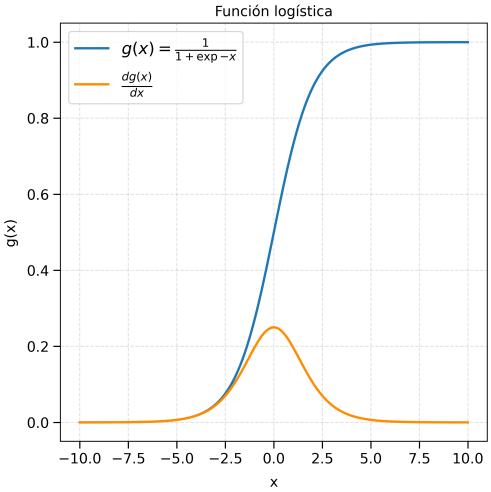
Bounded output: $0 < h_{\boldsymbol{\theta}}(\mathbf{x}) < 1.$

Natural probabilistic interpretation:

$$P(y = 1 \mid \mathbf{x}; \boldsymbol{\theta}) = h_{\boldsymbol{\theta}}(\mathbf{x}),$$
$$P(y = 0 \mid \mathbf{x}; \boldsymbol{\theta}) = 1 - h_{\boldsymbol{\theta}}(\mathbf{x}).$$

Smooth, S-shaped curve (sigmoid).
Acts as a “soft threshold” around $\boldsymbol{\theta}^\top \mathbf{x} = 0.$

Logistic function for classification



Useful property of the derivative:

$$
\begin{aligned}
g'(z) &= \frac{d}{dz}\frac{1}{1+e^{-z}} \\
&= \frac{1}{(1+e^{-z})^2}\left(e^{-z}\right) \\
&= \frac{1}{(1+e^{-z})}\left(1-\frac{1}{(1+e^{-z})}\right) \\
&= g(z)(1-g(z))
\end{aligned}
$$

Bernoulli probabilistic model

We assume a Bernoulli model:

$$P(y = 1 \mid \mathbf{x}; \boldsymbol{\theta}) = h_{\boldsymbol{\theta}}(\mathbf{x}),$$
$$P(y = 0 \mid \mathbf{x}; \boldsymbol{\theta}) = 1 - h_{\boldsymbol{\theta}}(\mathbf{x}).$$

Compact form (using $y \in \{0, 1\}$):

$$p(y \mid \mathbf{x}; \boldsymbol{\theta}) = \big(h_{\boldsymbol{\theta}}(\mathbf{x})\big)^{y} \big(1 - h_{\boldsymbol{\theta}}(\mathbf{x})\big)^{1-y}.$$

Likelihood function

For a dataset $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{n}$, assuming conditional independence:

$$
\begin{aligned}
L(\boldsymbol{\theta}) &= p(\mathbf{y} \mid \mathbf{X}; \boldsymbol{\theta}) \\
&= \prod_{i=1}^{n} p(y^{(i)} \mid \mathbf{x}^{(i)}; \boldsymbol{\theta}) \\
&= \prod_{i=1}^{n} \left(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})\right)^{y^{(i)}} \left(1 - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})\right)^{1-y^{(i)}}.
\end{aligned}
$$

Maximum likelihood estimator

Maximizing the log-likelihood is easier:

$$\ell(\boldsymbol{\theta}) = \log L(\boldsymbol{\theta}) = \sum_{i=1}^{n} \left[y^{(i)} \log h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})) \right].$$

Gradient ascent update:

$$\boldsymbol{\theta} := \boldsymbol{\theta} + \alpha \nabla_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}).$$

Gradient of the Log-Likelihood

(stochastic gradient ascent rule)

$$
\begin{aligned}
\frac{\partial}{\partial \theta_j} \ell(\boldsymbol{\theta}) &= \left(y \frac{1}{g\left(\boldsymbol{\theta}^\top \mathbf{x}\right)} - (1-y) \frac{1}{1 - g\left(\boldsymbol{\theta}^\top \mathbf{x}\right)} \right) \frac{\partial}{\partial \theta_j} g\left(\boldsymbol{\theta}^\top \mathbf{x}\right) \\
&= \left(y \frac{1}{g\left(\boldsymbol{\theta}^\top \mathbf{x}\right)} - (1-y) \frac{1}{1 - g\left(\boldsymbol{\theta}^\top \mathbf{x}\right)} \right) g\left(\boldsymbol{\theta}^\top \mathbf{x}\right) \left(1 - g\left(\boldsymbol{\theta}^\top \mathbf{x}\right)\right) \frac{\partial}{\partial \theta_j} \boldsymbol{\theta}^\top \mathbf{x} \\
&= \left(y \left(1 - g\left(\boldsymbol{\theta}^\top \mathbf{x}\right)\right) - (1-y) g\left(\boldsymbol{\theta}^\top \mathbf{x}\right)\right) x_j \\
&= \left(y - h_{\boldsymbol{\theta}}(\mathbf{x})\right) x_j
\end{aligned}
$$

$$
\therefore \theta_j := \theta_j + \alpha \left(y^{(i)} - h_{\boldsymbol{\theta}}\left(\mathbf{x}^{(i)}\right) \right) x_j^{(i)}
$$

error: $\left(\textbf{label} - \textbf{prediction}\right) \times$ **feature.**

Logistic loss definition

Define the logistic loss at the example level:

$$\ell_{\text{logistic}} : \mathbb{R} \times \{0, 1\} \to \mathbb{R}_{\geq 0},$$

$$\ell_{\text{logistic}}(t, y) \triangleq -\ell(\boldsymbol{\theta}) = y \log(1+e^{-t}) + (1-y) \log(1+e^{t}),$$

where $t = \boldsymbol{\theta}^\top \mathbf{x}$ is called the **logit**.

This is also known as “log loss” or “binary cross-entropy”.

Logistic loss gradient

(stochastic gradient ascent rule)

$$\frac{\partial \ell_{\text{logistic}}}{\partial \theta_j} = \left(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)}\right) x_j^{(i)}.$$

Logistic regression for multiple classes

Response variable $y \in \{1, 2, \dots, k\}$.

$p(y \mid \mathbf{x}; \boldsymbol{\theta})$ is a distribution over k possible discrete outcomes/classes (multinomial distribution).

Multinomial distribution involves k probabilities $\phi_1, \dots, \phi_k$, with $\sum_{i=1}^{k} \phi_i = 1$.

Goal: Build a model that, given $\mathbf{x}$, outputs $\phi_1, \dots, \phi_k$ as class probabilities.

Parameter groups for multiclass

Introduce k parameter vectors $\boldsymbol{\theta}_1, \dots, \boldsymbol{\theta}_k$, each $\boldsymbol{\theta}_i \in \mathbb{R}^D$.

Use the logits $\boldsymbol{\theta}_1^\top \mathbf{x}, \dots, \boldsymbol{\theta}_k^\top \mathbf{x}$ to represent probabilities, i.e., $P(y = 1 \mid \mathbf{x}; \boldsymbol{\theta}), \dots, P(y = k \mid \mathbf{x}; \boldsymbol{\theta})$.

Note that:

- $\boldsymbol{\theta}_j^\top \mathbf{x}$ is not restricted to $[0, 1]$.
- $\sum_{j=1}^{k} \boldsymbol{\theta}_j^\top \mathbf{x}$ does not sum to 1.

Solution: Apply the softmax function.

Softmax Function

The softmax function softmax : $\mathbb{R}^k \to \mathbb{R}^k$ is defined by:

$$\text{softmax}(t_1, \dots, t_k) = \begin{bmatrix} \dfrac{\exp(t_1)}{\sum_{j=1}^k \exp(t_j)} \\ \vdots \\ \dfrac{\exp(t_k)}{\sum_{j=1}^k \exp(t_j)} \end{bmatrix}$$

- Inputs t_i are called **logits**.
- Output is a probability vector (non-negative entries that sum to 1).
- Generalizes the sigmoid to the multiclass setting.

Softmax regression model

Define logits as $t_i = \boldsymbol{\theta}_i^\top \mathbf{x}$.

The softmax regression model is:

$$P(y = i \mid \mathbf{x}; \boldsymbol{\theta}) = \phi_i = \frac{\exp(\boldsymbol{\theta}_i^\top \mathbf{x})}{\sum_{j=1}^{k} \exp(\boldsymbol{\theta}_j^\top \mathbf{x})}.$$

Decision rule (0–1 loss):

$$\hat{y}(\mathbf{x}) = \arg\max_i \phi_i.$$

Negative log-likelihood (one example).

For a single example $(\mathbf{x}, y)$, the loss is

$$
\begin{aligned}
-\log p(y \mid \mathbf{x}; \boldsymbol{\theta}) &= -\log \left(\frac{\exp(t_y)}{\sum_{j=1}^{k} \exp(t_j)} \right) \\
&= -\log \left(\frac{\exp(\boldsymbol{\theta}_y^\top \mathbf{x})}{\sum_{j=1}^{k} \exp(\boldsymbol{\theta}_j^\top \mathbf{x})} \right) .
\end{aligned}
$$

Score of the correct class.

Sum of scores over all classes.

Total loss over the dataset

For n examples $(\mathbf{x}^{(i)}, y^{(i)})$ and k classes:

$$\ell(\boldsymbol{\theta}) = \sum_{i=1}^{n} -\log\left(\frac{\exp\left(\boldsymbol{\theta}_{y^{(i)}}^{\top}\mathbf{x}^{(i)}\right)}{\sum_{j=1}^{k}\exp\left(\boldsymbol{\theta}_{j}^{\top}\mathbf{x}^{(i)}\right)}\right).$$

This is the negative log-likelihood of the multinomial model (a multiclass generalization of the logistic loss).

Cross-Entropy Loss (Definition)

Define cross-entropy at the example level:

$$\ell_{\text{ce}} : \mathbb{R}^k \times \{1, \dots, k\} \to \mathbb{R}_{\geq 0},$$

$$\ell_{\text{ce}}((t_1, \dots, t_k), y) = -\log \left(\frac{\exp(t_y)}{\sum_{j=1}^{k} \exp(t_j)} \right).$$

- Inputs: logits $t_j = \boldsymbol{\theta}_j^\top \mathbf{x}$ and label y.
- Output: non-negative loss value.

Modular loss formulation

Using the cross-entropy loss notation, the total loss is:

$$\ell(\boldsymbol{\theta}) = \sum_{i=1}^{n} \ell_{\text{ce}}\big((\boldsymbol{\theta}_1^\top \mathbf{x}^{(i)}, \dots, \boldsymbol{\theta}_k^\top \mathbf{x}^{(i)}), y^{(i)}\big).$$

(sum over examples of "loss(logits, label)")

Cross-entropy gradient (w.r.t. logits)

Let

$$\phi_i = \frac{\exp(t_i)}{\sum_{j=1}^{k} \exp(t_j)}$$

be the softmax probability for class i.

Then, for each component t_i:

$$\frac{\partial \ell_{\text{ce}}(t, y)}{\partial t_i} = \phi_i - \mathbb{1}\{y = i\},$$

where $\mathbb{1}\{y = i\}$ is 1 if $y = i$, 0 otherwise.

Vector form of the gradient

In vector form:

$$\frac{\partial \ell_{\text{ce}}(t, y)}{\partial \mathbf{t}} = \boldsymbol{\phi} - \mathbf{e}_y,$$

- $\mathbf{e}_y \in \mathbb{R}^k$: y-th canonical basis vector (one-hot).
- $\boldsymbol{\phi} \in \mathbb{R}^k$: predicted probability vector.

Parameter gradients

For one example $(\mathbf{x}, y)$:

$$\frac{\partial \ell_{\text{ce}}}{\partial \boldsymbol{\theta}_i} = \frac{\partial \ell_{\text{ce}}}{\partial t_i} \frac{\partial t_i}{\partial \boldsymbol{\theta}_i} = \left(\phi_i - \mathbb{1}\{y = i\}\right) \mathbf{x}.$$

For all examples $(\mathbf{x}^{(j)}, y^{(j)})$, $j = 1, \ldots, n$:

$$\frac{\partial \ell(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}_i} = \sum_{j=1}^{n} \left(\phi_i^{(j)} - \mathbb{1}\{y^{(j)} = i\}\right) \mathbf{x}^{(j)}.$$

Gradient descent updates

Gradient descent update for each class i:

$$\boldsymbol{\theta}_i \leftarrow \boldsymbol{\theta}_i - \eta \frac{\partial \ell(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}_i},$$

where $\eta > 0$ is the learning rate.

In practice:

- Use mini-batch stochastic gradient descent.
- Optionally add regularization, e.g. $+\lambda \|\boldsymbol{\theta}\|^2$.